EvOLVe

Vinaayak Kanagaraj

February 2026

1 Introduction

TThis paper proposes a fitness-modulated, trajectory-aware evolutionary algorithm for adaptive game-playing agents, in which hyperparameter updates are guided by fitness history, semantic interpretation of the environment, and similarity-aware population control rather than a fixed objective function.

Across the literature, recurring limitations include:

- Optimization in fixed, stationary environments,
- Manual fitness design and static reward formulations,
- Convergence toward static optima rather than adaptive stability,
- Limited cross-game benchmarking,
- Weak analysis of recovery under environmental perturbations,
- Partial parameter tuning instead of full rule-level co-evolution,
- Human-defined search boundaries,
- Absence of integrated agent–environment feedback loops,
- Scalability challenges as hyperparameters expand.

1.1 Phase 1 — Empirical Strategy Optimization and Benchmarking

The first phase focuses on empirical strategy optimization using standard evolutionary algorithms (EAs), evaluating convergence rate, fitness progression, win-rate dynamics, and learning speed across multiple test cases.

Several research papers, such as *Evolutionary Computation and Games* (Lucas & Kendall) explore EA applications across games and identify convergence behaviors. However, they stop short of defining standardized and dynamic benchmarking protocols. *Strategy Optimization with Its Application to Game Theory* focuses on convergence in structured game models using hill-climbing,

but assumes stationary payoff environments and does not examine adaptive recovery under slightly modified test cases.

The authors of *Optimizing Player Behavior in a Real-Time Strategy Game* and *Optimizing Strategy Parameters in a Game Bot* gave focus to GA-based tuning and mutation sensitivity analysis. However, these studies utilize single-game, fixed-rule settings, which risks overfitting. More recent works such as *Efficient Evolutionary Methods for Game Agent Optimisation* and *Evolutionary Algorithm for Parameter Optimization in Games* improve computational efficiency and real-valued optimization, but have the disadvantage of being largely offline and reward-static.

Similarly, *Evolutionary Game Algorithm for Continuous Parameter Optimization* connects evolutionary game theory and optimization but does not address real-time adaptation under environmental shifts.

Our Phase 1 aims to focus on a better understanding of these gaps and their occurrences. We employ benchmarking across various factors like sensitivity to modified environmental hyper-parameters, static fitness functions and convergence rate.

1.2 Phase 2 — Co-Evolution and Predator–Prey Competitive Agents

The second phase will focus on competitive co-evolution via a Predator–Prey framework, where selection, crossover, and mutation drive sustained arms-race pathways. These pathways will be multi-objective to ensure maintenance of diversity.

*T*he paper Coevolution Versus Self-Play Temporal Difference Learning highlights instability and continuous-cycling in direct implementations of co-evolution. *Multi-objective Competitive Co-evolutionary Optimization* states that multiple optimization objectives prevent one agent from dominating others and wiping out diversity. These findings are further supported by the sources: *Constructing Competitive and Cooperative Agents using Multi-Component Enforced Subpopulations*, *Using a Training Camp with Genetic Programming to Evolve Game Players*, and *Evolving Combat Algorithms to Control Space Ships in a Video Game*.

1.3 Phase 3 — Hyperparameter and Environment Co-Evolution

The final phase extends evolution to the environment itself, treating tunable parameters (resource constraints, scoring rules, difficulty, rewards, penalties, learning rates) as co-evolving entities. *Evolutionary Algorithms for a Better Gaming Experience* explores dynamic difficulty adjustment, but focuses mainly on scaling difficulty rather than full environment evolution. *An Evolutionary Approach to Balancing and Disrupting Gameplay* and *Using Evolutionary Algorithms to Target Complexity in Games* study gameplay balancing and complexity control,

yet treat parameter tuning as a separate optimization task rather than a co-evolving process. *Supporting Game Design with Evolutionary Algorithms* offers EA-based design assistance, but remains guided by designer-defined constraints. *Game Theory-Inspired Evolutionary Algorithm for Optimization* incorporates rule adaptation, though without large-scale agent–environment co-evolution. Finally, *General Video Game AI: A Multitrack Framework for Evaluating Agents* supports cross-game evaluation, but evolves agents only, not the environments themselves.

2 Mathematical and Logical Basis

Algorithm 1: Fitness-Modulated Hyperparameter Trajectory Evolution

Initialize population $\mathcal{P}_0$, archive $\mathcal{A}$, encoder NN_{env};
for $t = 1, 2, \ldots$ **do**
  Evaluate fitness f_i^t and update trajectory τ_i^t for all $\mathcal{O}_i$;
  Compute expressibility $e_i^t = \sigma(\sum_k \alpha_k f_i^{t-k})$;
  Encode environment $\mathbf{z}_t = NN_{env}(state_t)$;
  Update hyperparameters $\lambda_i^{t+1} = \lambda_i^t + e_i^t g(\mathbf{z}_t, \tau_i^t) + \epsilon_i^t$;
  Remove $\mathcal{O}_i$ if $\bar{f}_i^t < \delta_f$;
  Remove redundant $\mathcal{O}_j$ if $S(i,j) > \delta_s$;
  Insert survivors into $\mathcal{A}$ and mutate offspring;

3 Combinational Novelty: Trajectory-Semantic Co-Evolution

3.1 Definition of the Framework

We introduce a class of evolutionary algorithms referred to as **Trajectory-Semantic Co-Evolutionary Algorithms (TSCEA)**. This framework integrates three key mechanisms: fitness-history modulation, semantic encoding of the environment, and similarity-constrained hyperparameter adaptation.

Let $\mathcal{P}_t = \{\mathcal{O}_i^t\}_{i=1}^N$ denote the population at generation t. Each organism $\mathcal{O}_i^t$ is defined as:

$$\mathcal{O}_i^t = \left(\theta_i^t, \lambda_i^t, \tau_i^t\right)$$

where:

- θ_i^t represents strategy parameters,
- λ_i^t denotes hyperparameters,
- $\tau_i^t = \{f_i^{t-k}\}_{k=0}^K$ is the fitness trajectory.

3.2 Fitness-Trajectory Modulation

Rather than optimizing a single static objective value, we model performance through a trajectory-based measure defined as:

$$e_i^t = \sigma\left(\sum_{k=0}^{K} \alpha_k f_i^{t-k}\right)$$

where $\sigma(\cdot)$ denotes a bounded activation function and α_k represent temporal weighting coefficients.

This formulation introduces memory-aware selection pressure, shifting evolution away from purely instantaneous fitness evaluation.

3.3 Semantic Environment Conditioning

Let s_t denote the environment state at time t. We compute a semantic embedding of this state as:

$$z_t = \mathcal{E}(s_t)$$

where $\mathcal{E} : \mathcal{S} \to R^d$ is a learned environment encoder.

Unlike traditional EA formulations, environmental context modulates hyperparameter evolution.

3.4 Hyperparameter Drift Dynamics

Hyperparameters evolve according to:

$$\lambda_i^{t+1} = \lambda_i^t + e_i^t \, g(z_t, \tau_i^t) + \epsilon_i^t$$

where:

- $g(\cdot)$ denotes a function capturing the interaction between fitness trajectory and environment state,
- $\epsilon_i^t \sim \mathcal{N}(0, \sigma^2)$ represents stochastic mutation noise.

Accordingly, hyperparameter evolution becomes:

1. Dependent on fitness history,
2. Conditioned on the environment,
3. Subject to stochastic perturbation.

3.5 Similarity-Constrained Population Control

To reduce degeneracy and avoid premature convergence to local optima, we introduce a similarity metric defined as:

$$S(i, j) = d(\theta_i^t, \theta_j^t)$$

Redundant individuals are removed if:

$$S(i, j) > \delta_s$$

This enforces diversity as an explicit evolutionary constraint, rather than relying on it to emerge implicitly.

3.6 Unified Evolutionary Operator

The complete TSCEA update operator is:

$$\Phi : (\mathcal{P}_t, s_t) \mapsto \mathcal{P}_{t+1}$$

where

$$\Phi = \mathcal{M} \circ \mathcal{D} \circ \mathcal{H} \circ \mathcal{F}$$

with:

- $\mathcal{F}$: fitness-trajectory evaluation,
- $\mathcal{H}$: semantic hyperparameter drift,
- $\mathcal{D}$: diversity-based pruning,
- $\mathcal{M}$: mutation and offspring generation.

3.7 Combinational Novelty Claim

Proposition (Combinational Novelty). TSCEA differs from classical evolutionary algorithms in that hyperparameter updates are jointly influenced by:

1. Multi-step fitness trajectory memory,
2. Learned semantic environmental embedding,
3. Explicit similarity-constrained population regulation.

While existing adaptive and self-adaptive evolutionary strategies incorporate some of these mechanisms individually, TSCEA integrates them within a unified update framework. This shifts the focus of evolutionary optimization from static objective maximization toward trajectory-aware and environment-informed adaptation.